Bio:

- Name: Navid Shabani Barzegar
- Hometown: Kingston, Ontario, Canada
- LinkedIn: https://www.linkedin.com/in/navid-shabani-barzegar-16036a35a/

DaT Parkinson's Challenge: model documentation and write-up (team NavAttack, 2nd place)

Prize submission: "v17", DrivenData submission id 322397. Public log loss 0.2409, private log loss 0.2731, private AUROC 0.9524.

Summary

This model reads one DaT-SPECT brain scan and returns the probability that a nuclear-medicine reader would report it as abnormal. It first rescales each scan against its own non-specific brain uptake, so that striatal values from different cameras share one scale. Four families of models then combine their outputs. Regression models read measurements of the caudate and putamen, such as the ratio of posterior to anterior putamen uptake. A 3D neural network reads the striatal region directly and scores each hemisphere separately, so that loss on one side is enough for an abnormal call. 2D image networks read projections of the same region. A tabular model compares each scan with the training cases. On the hidden test set it scored log loss 0.2731 and AUROC 0.952. It has no clinical validation.

1. Who are you (mini-bio) and what do you do professionally?

Team NavAttack has one member.

I'm Navid Shabani Barzegar, a fourth-year Honours Health Sciences student at Queen's University in Kingston, Canada, graduating in 2027. I hope to study medicine and build a career in neuroscience, with a particular interest in understanding how the brain computes. The brain is still the organ we understand least, and I believe machine learning will change how quickly that gap closes. I built and trained this solution on my laptop over the summer.

2. What motivated you to compete in this challenge?

I wanted to apply machine learning to a clinical question about the brain and see how far careful work could go. This challenge offered clinical scans from many cameras and sites, and it scored every entry on a test set that no participant could see. That variation interested me as much as the disease itself, and much of my work went into separating acquisition effects from disease signal. The challenge also let me take one problem

through every stage myself, from the raw scan to a calibrated probability, and find out which of my choices held up.

3. High level summary of your approach: what did you do and why?

v17 applies per-scan corrections and then combines four model families with fixed logit weights.

Preprocessing. A log-intensity Otsu threshold gives the head mask. A disc guard re-thresholds masks larger than 7.5 L, which cut the loss of one centre with noisy reconstruction discs from 0.1733 to 0.1307. The reference level is the mode of log-intensity inside the mask eroded by 10 mm, without the top 8% of voxels. My first submission used a percentile mask that swallowed the scatter halo on large fields of view. Fixing it raised the single-feature AUROC of striatal specific-binding ratio (SBR) from 0.56 to 0.84. A two-peak localiser finds both striata and sets a canonical frame, from which I sample a 96 mm cube at 2 mm (48^3 voxels) in SBR units. For the feature models I register that crop rigidly to a template built from training crops. This removed a median residual rotation of 10.3° and moved stacked CV loss from 0.2274 to 0.2206.

Members. (1) Two L1 logistic regressions read 235 and 94 hand-crafted region features. (2) A 3D hemisphere multiple-instance CNN scores each hemisphere with a shared encoder and combines them by noisy-OR. In a same-seed fold-0 test it scored 0.2543 against 0.2825-0.2916 for whole-crop runs. I average 34 checkpoints. (3) Three ImageNet-pretrained EfficientNet models (two B0 seeds and one B3) read three maximum-intensity projections of a second crop in a guarded frame. (4) A TabICLv2 in-context model reads 64 selected features, 4 acquisition descriptors and 4 linear prior scores. Each prior is cross-fitted with its acquisition family held out (an acquisition family is a coarser grouping of the inferred centres), and two priors add scale-space shape and spatial Gaussian-mixture descriptors.

Combination. Members (1) to (3) enter a fixed-ratio logit stack, which beat a fitted stacker on 6 of 6 re-splits. I chose temperature T = 1.4 from the public temperature curve (0.2611 at T = 1.0, 0.2507 at 1.2 and about 0.2502 at 1.4). The last point comes from v11, which also changed the 3D member and the stacker. The final logit is $z = 0.65 \times \text{logit}(p_{stack}) / 1.4 + 0.35 \times \text{logit}(p_{tab}) / 1.2$. A scan whose registration fails the quality check (NCC below 0.30 or an implausible rotation) receives the training prior 0.5485, and I clip outputs to [0.005, 0.995].

Locally the first TabICL block moved the ensemble from 0.2167 to 0.2077 (paired 95% interval -0.0147 to -0.0062), and the v16 and v17 tabular changes moved it to 0.2039.

4. Do you have any useful charts, graphs, or visualizations from the process?

The five figures below are also in `docs/figures/` of the code archive. `make_figures.py` in the same folder draws them from aggregate numbers only.

Pipeline (`pipeline_diagram.png`, vector copy `pipeline_diagram.svg`). The v17 path from one NIfTI file to one probability.

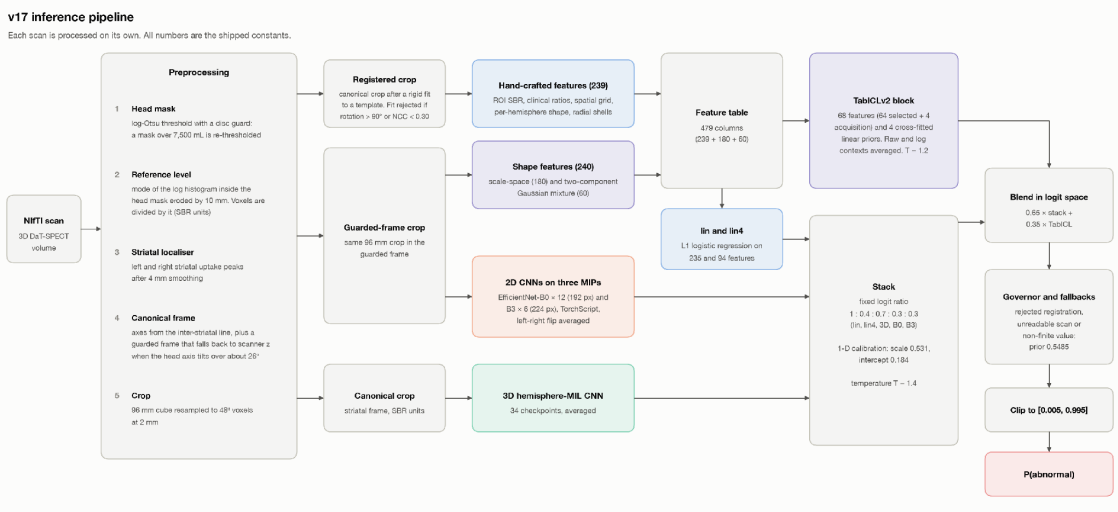


Figure 1. The v17 inference pipeline

Score progression (`score_progression.png`). Public and private log loss of the nine scored submissions in upload order, each labelled with its change. The reference-level fix (v2 to v3) is the largest step.

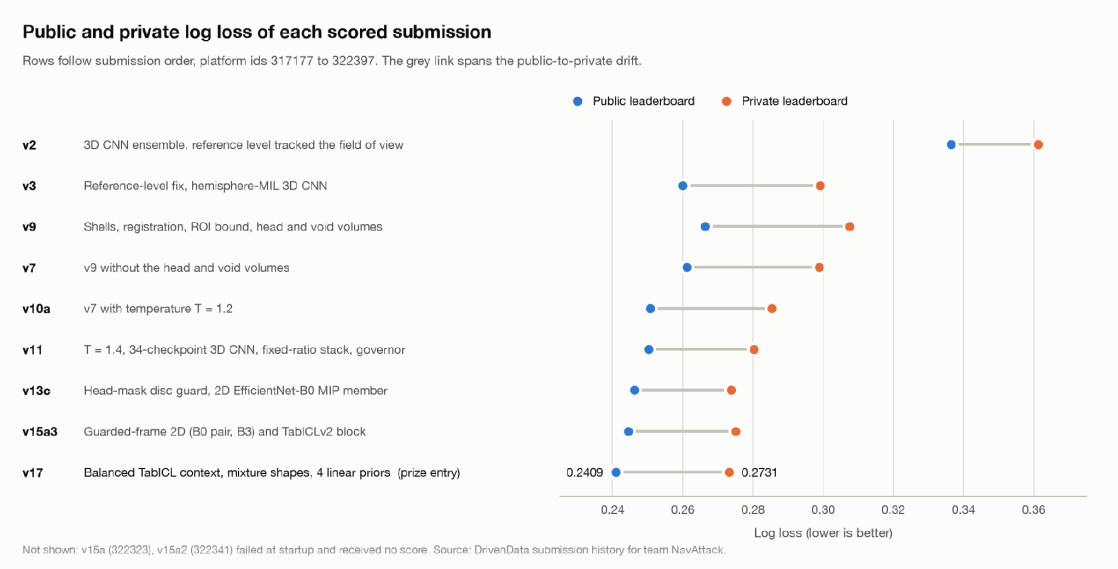


Figure 2. Public and private log loss of each scored submission

Drift (`drift.png`). Private minus public log loss per submission. Every version scored worse on the private slice, by 0.0248 (v2) to 0.0411 (v9), and v17 by 0.0322.

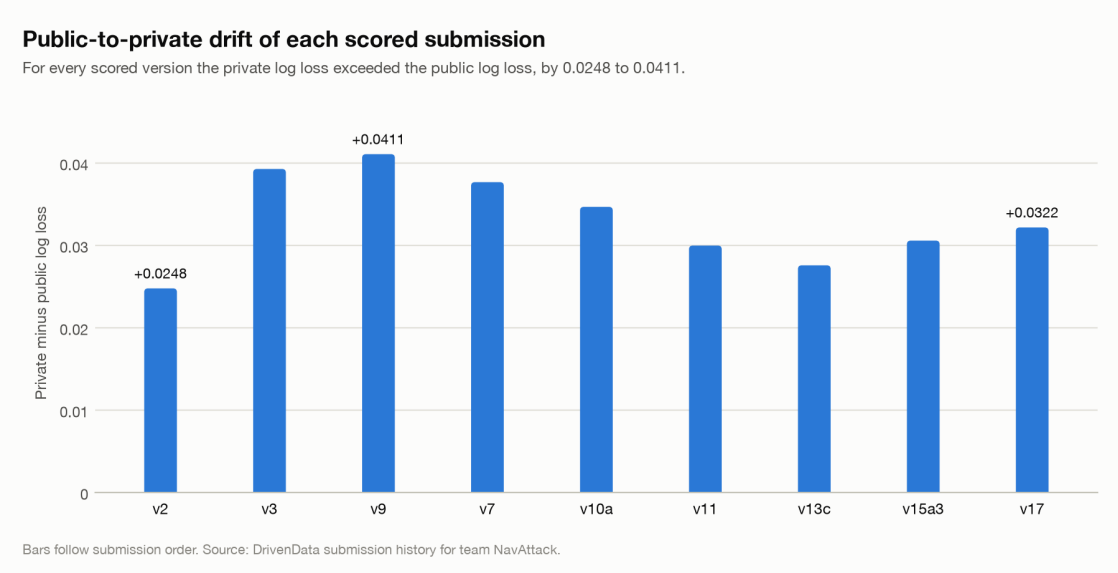


Figure 3. Public-to-private drift of each scored submission

Loss by inferred centre (`family_loss.png`). Scan count, prevalence, out-of-fold log loss and AUROC for six inferred centres. The predictions come from an earlier stack at T = 1. The two hardest inferred centres hold 36% of scans and 45% of out-of-fold loss.

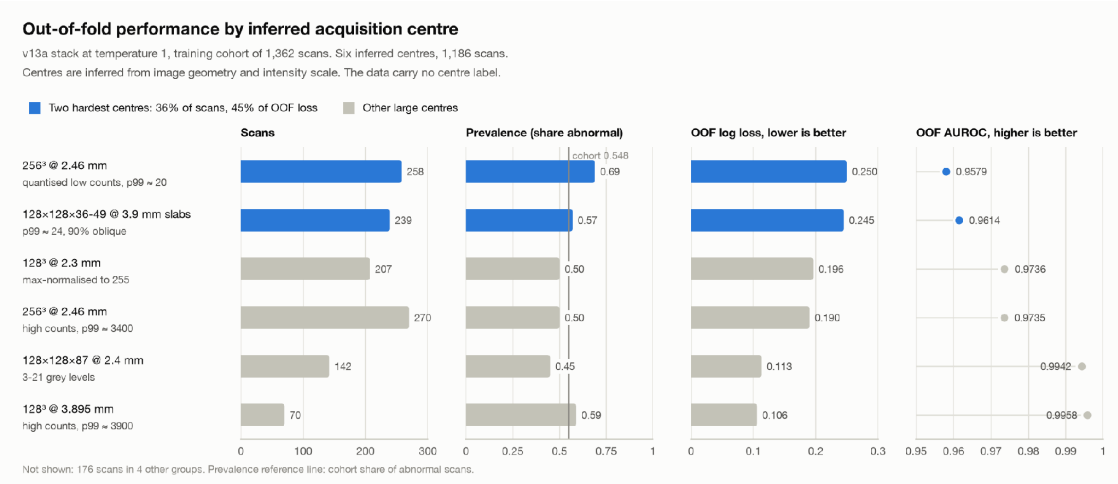


Figure 4. Out-of-fold performance by inferred centre

Linear coefficients (`lin_coefficients.png`). The 15 largest standardised coefficients of the shipped L1 model `lin` (section 6).

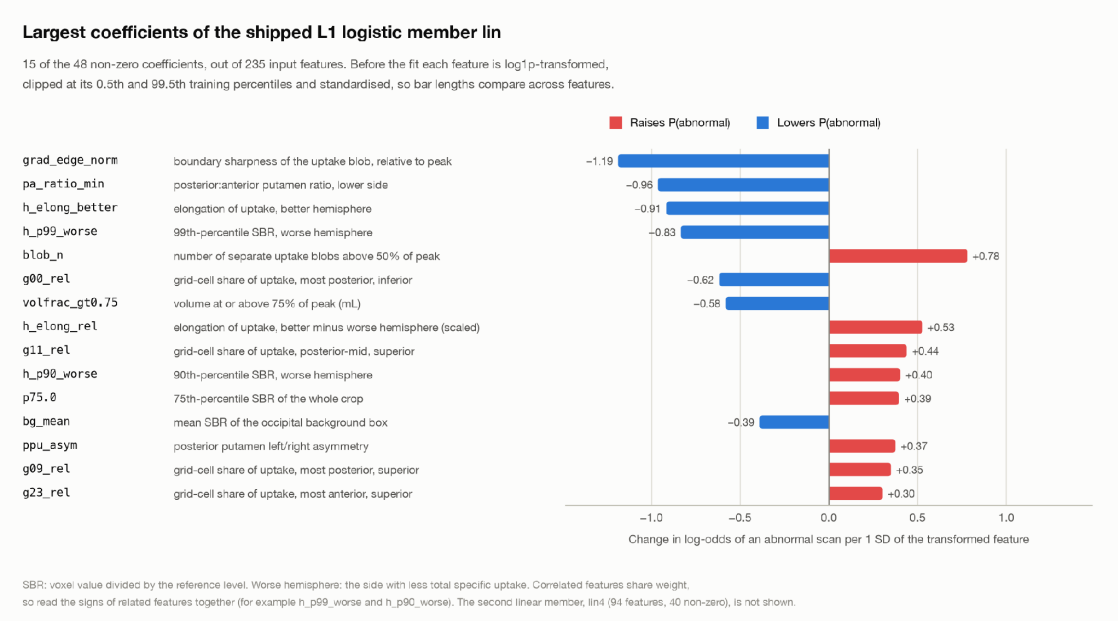


Figure 5. Largest coefficients of the shipped L1 logistic member lin

5. The three most impactful code parts

Question: Copy and paste the 3 most impactful parts of your code and explain what each does and how it helped your model.

I copied all three excerpts unchanged from the shipped inference code (`inference/`), which is byte-identical to the scored v17 archive.

5.1 Head mask and reference level (`inference/preprocessing3.py`)

`inference/preprocessing3.py:187-198`

```
    pos = small[small > 0]
    if pos.size < 64:
        z = np.zeros_like(small, dtype=bool)
        return (z, z) if return_holes else z
    lv = np.log(pos + 1e-3)
    # trim the extreme tails so a few hot/cold voxels cannot drag the
threshold
    lo, hi = np.percentile(lv, [1.0, 99.5])
    lv = lv[(lv >= lo) & (lv <= hi)]
    thr = np.exp(_otsu(lv)) - 1e-3
    m = small > max(thr, 0.0)
    m = ndimage.binary_closing(m, iterations=1)
    m = ndimage.binary_fill_holes(m)
```

`inference/preprocessing3.py:256-284`

```python
def reference_from_mask(small: np.ndarray, mask: np.ndarray) -> float:
    """Mode of log-intensity inside the head, striatum excluded.

    Mode (not median) because the head mask may over-include a scatter
rim; a
    median shifts with that contamination while the mode stays pinned
to the
    dominant non-specific tissue compartment.
    """
    vals = small[mask]
    vals = vals[vals > 0]
    if vals.size < 32:
        pos = small[small > 0]
        return float(np.median(pos)) if pos.size else 1.0
    # drop the specific-binding tail (striatum is ~1-3% of head
volume)
    cut = np.percentile(vals, 92.0)
    body = vals[vals <= cut]
    if body.size < 32:
        body = vals
    lv = np.log(body + 1e-3)
    lo, hi = np.percentile(lv, [2.0, 98.0])
    if hi <= lo:
        return float(np.median(body))
    hist, edges = np.histogram(lv, bins=96, range=(lo, hi))
    # light smoothing so the mode is not a sampling artefact
    hist = ndimage.uniform_filter1d(hist.astype(np.float64), 5)
    centers = 0.5 * (edges[:-1] + edges[1:])
    ref = float(np.exp(centers[int(np.argmax(hist))]) - 1e-3)
    if not np.isfinite(ref) or ref <= 0:
        ref = float(np.median(body))
    return max(ref, 1e-6)
```

What it does. This code sets the denominator of every SBR value. The log-intensity histogram is bimodal (air and scatter against tissue) at every field of view, so an Otsu split in log space finds the tissue floor. Before `reference_from_mask` runs, `inner_mask` (`preprocessing3.py:244-253`, called at lines 628 and 723) erodes the mask by 10 mm, backing off to 6 or 3 mm if less than 300 mL would remain. `reference_from_mask` drops the top 8% of voxels, which hold the striatum, and returns the mode of the smoothed histogram. The mode stays on non-specific tissue even when the mask includes some scatter.

How it helped. My v2 mask kept voxels above the 12th percentile of non-zero voxels. On large 256^3 fields of view it took in the scatter halo, so the reference tracked the field of view. Striatal SBR spanned 3.5 to 300 (85-fold), its within-normal coefficient of variation was 1.28 and its single-feature AUROC was 0.56. After the fix these became 2.3 to 36, 0.23 and 0.84. Public log loss fell from 0.3364 (v2) to 0.2599 (v3), and private

from 0.3612 to 0.2992. v3 also added the hemisphere CNN and leak-free out-of-fold predictions, so only the AUROC change isolates this fix.

5.2 Hemisphere multiple-instance CNN with noisy-OR (`inference/nets3.py`)

`inference/nets3.py:132-149`

```
    def hemi_logits(self, x):
        """Per-hemisphere logits. Each is the log-odds that THAT
hemisphere is
        intact; the head learns the sign, and `forward` combines them
so that the
        scan reads abnormal as soon as either side is not intact."""
        n = x.shape[-3]
        h = n // 2
        left = torch.flip(x[..., :h, :, :], dims=[-3])      # mirror
to right-handed
        right = x[..., n - h:, :, :]
        return self._score(left), self._score(right)

    def forward(self, x):
        zl, zr = self.hemi_logits(x)
        # Noisy-OR over the two hemispheres, in log space for
stability.
        # With qL = sigmoid(zl) = P(left intact),  P(abnormal) = 1 -
qL*qR.
        #   log(qL*qR) = -softplus(-zl) - softplus(-zr)      [log
sigmoid(z) = -softplus(-z)]
        neg = -(F.softplus(-zl) + F.softplus(-zr))          # log
P(both intact)
        pos = torch.log1p(-torch.exp(neg.clamp(max=-1e-6)))  # log
P(abnormal)
        return pos - neg                                     # logit
P(abnormal)
```

What it does. The crop splits at the mid-sagittal plane of the canonical frame. The network mirrors the left half and passes both halves through one shared 3D encoder. The whole network has 896,309 trainable parameters, counted from the shipped checkpoint. Each half gets a logit for its hemisphere being intact, and `forward` combines them as a noisy-OR, P(abnormal) = 1 - qL × qR. One depleted side is enough for an abnormal call, which matches the asymmetric onset of Parkinson's disease. The 1,362 scans give 2,724 training hemispheres.

How it helped. In a same-seed fold-0 test at 32^3 the noisy-OR network scored log loss 0.2543, against 0.2825-0.2916 for three whole-crop runs. At 48^3 single five-fold runs score 0.2606-0.2667, and the six-run average scores 0.2600 (AUROC 0.9585). The network is invariant to a left-right flip, so v17 skips mirror test-time augmentation.

5.3 Tabular block: prior scores, TabICL and the final blend (`inference/tabular_adapter.py`)

`inference/tabular_adapter.py:53-72`

```
    def predict(self,full_features):
        if len(full_features)==0:return np.zeros(0,dtype=np.float64)
        X=np.asarray(full_features,dtype=np.float64)[:,self.order]
        if self.priors:
            full=np.asarray(full_features,dtype=np.float64)

scores=np.column_stack([prep.transform(full[:,idx])@coef+intercept for
idx,prep,coef,intercept in self.priors])
            X=np.column_stack([X,scores])
        X=np.where(np.isfinite(X),X,self.median)
        p=np.mean([model.predict_proba(self._transform(X,kind))[:,1]
for kind,model in self.models],axis=0)
        if not np.isfinite(p).all():raise RuntimeError('Tabular
inference returned an invalid probability')
        return p.astype(np.float64)

    def blend(self,legacy_probability,tabular_probability):
        eps=1e-6
        def logit(p):
            p=np.clip(np.asarray(p,dtype=np.float64),eps,1-eps)
            return np.log(p/(1-p))

w=float(self.spec['blend_weight']);T=float(self.spec['temperature'])
        z=(1-w)*logit(legacy_probability)
+w*logit(tabular_probability)/T
        return 1/(1+np.exp(-np.clip(z,-30,30)))
```

What it does. `predict` appends four prior scores to 64 selected features and 4 acquisition descriptors. The priors are L1 logistic models on 235, 94, 415 (adding 180 scale-space shape columns) and 295 (adding 60 spatial Gaussian-mixture columns) features. Two frozen TabICLv2 contexts each return a probability, and the adapter averages them. The raw context holds all 1,362 training cases, and the log-feature context an acquisition-balanced subset of 1,148. Inside the contexts, each row's prior scores come from models fitted with that row's acquisition family held out. `blend` gives the tempered stack logit weight 0.65 and the tabular logit, divided by 1.2, weight 0.35.

How it helped. Locally the first TabICL block (v15a) moved the ensemble from 0.2167 to 0.2077 (paired bootstrap -0.0104, 95% interval -0.0147 to -0.0062). The v17 descriptors and priors moved the tabular component from 0.2372 to 0.2230 with each acquisition family held out (95% interval -0.0235 to -0.0047) and the ensemble from 0.2068 to 0.2039. The leaderboard gain was smaller. From v13c to v17, a span that also includes an unscored change to the 2D members, public log loss fell from 0.2462 to

0.2409 and private from 0.2738 to 0.2731. The first TabICL package (v15a3) beat v13c on the public set (0.2444) and lost to it on the private set (0.2750).

6. What features or areas of the image were most important to your performance?

The signal sits in the striatum, most strongly in the posterior putamen and in the more affected hemisphere.

- **Single features.** Posterior-putamen SBR has AUROC 0.84, the strongest column of the 94-column clinical table. Specific binding in the 6-9 mm shell around the worse hemisphere's uptake centroid has AUROC 0.840.
- **Rostro-caudal gradient.** In the registered frame, abnormal scans lose 22.6% of caudate uptake (AUROC 0.672) and 46.3% of posterior-putamen uptake (AUROC 0.856) relative to normal scans.
- **Shipped linear model.** `lin` keeps 48 of its 235 columns. Its largest standardised coefficients (negative means a lower value raises the probability of an abnormal scan):

feature	meaning	coefficient
`grad_edge_norm`	mean gradient on the edge of the region above 50% of peak uptake, divided by the peak	-1.19
`pa_ratio_min`	posterior-to-anterior putamen uptake ratio, lower of the two sides	-0.96
`h_elong_better`	elongation of the better hemisphere's high-uptake region	-0.91
`h_p99_worse`	99th-percentile uptake of the worse hemisphere	-0.83
`blob_n`	number of separate regions above 50% of peak uptake	+0.78
`g00_rel`	share of striatal uptake in the posterior, inferior cell of a 3 × 4 × 2 grid	-0.62

feature	meaning	coefficient
`volfrac_gt0.75`	volume above 75% of peak uptake	-0.58

L1 selection spreads weight unevenly across correlated columns, so these weights describe the fitted model and do not rank anatomy.

- **Image members.** The CNNs see only the 96 mm striatal crop: the 3D network in the canonical frame, the 2D networks as three maximum-intensity projections of a second crop in a guarded frame. Uptake outside the striatum scored AUROC 0.585 under leave-site-out validation and behaved as a site marker, so I left it out.
- **Acquisition descriptors.** The linear members exclude native voxel spacing, reference level, eroded-mask volume and head volume, because those columns identify the centre. The TabICL block reads all four.

7. Machine specs and run time

Question: Please provide the machine specs and time you used to run your model.

All development and training ran on one laptop.

- **CPU (model):** Apple M3 Pro, 11 cores (5 performance, 6 efficiency)
- **GPU (model):** Apple M3 Pro integrated 14-core GPU through PyTorch MPS, for training only. No CUDA. The scored inference ran on DrivenData's NVIDIA A100 80 GB container (24 vCPU, 220 GB RAM).
- **Memory (GB):** 18 GB unified memory
- **OS:** macOS 27.0 (build 26A428) for the timed retrain. Development and training ran on macOS 27 beta releases from July to September 2026.
- **Train duration:** 6 h 10 min (22,199 s) for all shipped models. I measured it on 27-28 September 2026 with one run of `pipeline/train_all.sh` (`MODE=full`). The run used a clean copy of the released code and raw competition data, on the otherwise idle laptop. It excludes exploration.

component	measured time
data check, folds, registered feature tables, linear members, crop caches	1,305 s
3D CNN: six five-fold runs and four full-data models	11,791 s
EfficientNet-B0: two five-fold runs and two full-data models	3,735 s
EfficientNet-B3: one five-fold run and one full-data model	4,860 s

component	measured time
ImageNet backbone check and TorchScript export	165 s
TabICL block: shape and mixture features, recipe, context freeze	301 s
stacker and packaging	42 s
total	**22,199 s**

The run used the released registration template. Rebuilding it from the run's own crop cache takes another 8-9 min (484 s and 518 s in two reruns) and gives the shipped template byte for byte.

- **Inference duration:** on the M3 Pro CPU, 150 scans took 497-530 s on an otherwise idle machine (runs on 14, 25 and 28 September 2026, 3.3-3.5 s per scan) and 13-17 min while other jobs shared the machine (up to 6.8 s per scan). Loading the models adds about 35 s once per run. On DrivenData's A100 container the scored run of submission 322397 took 12 min 57 s for the whole test set: 20 s to load the models and 12 min 36 s of inference (platform log, 14 September 2026).

8. Anything we should watch out for or be aware of in using your model (for example code quirks, memory requirements, numerical stability issues)?

- **Environment.** Use `requirements-inference.txt` (Python 3.12, torch 2.12.1, numpy 2.2.6, scikit-learn 1.8.0). The TabICL contexts hold pickled scikit-learn and numpy objects frozen under those versions. The adapter silences version warnings, so other versions could change results without a message. `main.py` exits with code 1 if psutil or huggingface-hub is missing.
- **Devices.** Inference uses CUDA if available, otherwise CPU, and never MPS.
- **Environment variables.** Fourteen `DAT_*` variables change preprocessing and features at inference, and two more (`DAT_DEVICE`, `DAT_NO_PRETRAINED`) affect training only. Leave all of them unset.
- **Silent fallbacks.** The code reads only `<uid>.nii.gz` files. A scan with another name or a failed preprocessing step receives the training prior 0.5485. The pipeline works in blocks of 128 scans. A registration error sends the whole block to the canonical frame with the governor off. A CNN error drops that member and all later ones for the block. Each fallback logs one static warning per run. TabICL has no fallback, and an error there stops the run.
- **Logging.** Log lines are static strings, apart from installed package versions and, on a crash, the exception type name. No test-scan value, identifier, count or path reaches the log.

- **Numerical differences.** The unmodified archive reproduced its 14 September predictions on the 150-scan local set exactly (difference 0.0) in an environment pinned to the platform's package versions, and to 4.0e-6 with newer versions. The pinned environment used scipy 1.17.1 in place of the platform's 1.15.3, whose macOS arm64 wheels do not load. The code registers scans and runs the CNNs in blocks of up to 128, so floating-point sums inside those batched kernels depend on which scans share a block. Shuffling the order of the 150 scans changed predictions by 9e-6 on average and by at most 1.3e-3 on one scan. Predicting five scans one at a time changed them by at most 3.1e-4. A later check in a fresh environment found 1.5e-5 for one scan alone against the same scan in a 20-scan batch, and 4.4e-5 for a two-scan folder. On the normal path no scan's prediction uses another scan's data: the registration objective is a sum of per-scan terms, the rejection threshold is a fixed per-scan NCC value, and all normalisation statistics are frozen from training. The exception is the error fallbacks above, which act on a whole block of up to 128 scans. None of their warnings appeared in the local runs or in the platform log of submission 322397. I never ran v17 on a local NVIDIA GPU, and a registration fit near the NCC threshold could flip between devices.
- **Retraining.** A complete retrain from the raw data (`MODE=full`, 28 September 2026) on the same laptop reproduced every CNN checkpoint tensor for tensor. The stacker constants came out bit for bit, and the rebuilt package's predictions on the 150-scan local set were identical to the scored package's (maximum absolute difference 0.0). The training code mixes numpy and device-side torch random draws, so on CUDA or the CPU expect weights within the seed spread (per-run out-of-fold log loss within about 0.001-0.003) rather than identical ones.
- **Train and serve crops.** I trained the 3D member on crops made before the disc guard existed, and inference applies the guard. The guard changes 29 of 1,362 training crops. Serving guarded crops to these checkpoints lowered one run's out-of-fold loss from 0.2639 to 0.2629. I fitted the 3D member's stack weight on a four-run out-of-fold file, while the shipped member averages six runs and four full-data models.
- **Memory.** The archive is 1.09 GB, including 457 MB of compressed TabICL contexts. Peak resident memory on the M3 Pro CPU (measured with `/usr/bin/time -l`) was 2.2 GB for one scan and 2.7-5.2 GB for runs of 20 to 150 scans, the upper end on an idle machine with memory to spare. For 150 scans macOS also reported peak memory footprints of 4.8-5.0 GB, which count compressed pages. Scans are processed in blocks of up to 128, so allow about 8 GB.
- **Published weights.** The 18 TorchScript files in the scored archive held debug records with a local file path. The released weights bundle (`navattack_v17_weights.zip`) ships copies without those records. They give bit-identical outputs, and `models/SHA256SUMS.scored.txt` and `models/SHA256SUMS.public.txt` list both sets of hashes.
- **Stale comment.** A comment in `inference_main.py` says T is 1.0. The configuration uses T = 1.4.

9. Did you use any tools for data preparation or exploratory data analysis that aren't listed in your code submission?

All data preparation and exploration used Python (NumPy, SciPy, pandas, nibabel, matplotlib). The repository includes the data survey (`src/eda_real.py`) and the NIfTI header survey (`tools/hdr_fingerprint.py`). I also used rendering and QA scripts that are not in the repository. They drew crops, the worst-scoring scans per acquisition group and result plots (`render_family.py`, `render_rg1.py`, `qa_v3.py`, `viz_crops.py`, `lookhard.py`, `hardcore.py`, `make_results3.py`). Two feature-table filtering steps first ran as one-off pandas commands and are now `src/filter_feature_tables.py`. I used macOS Preview to view images.

I used Claude Code and OpenAI Codex to help write and run analysis and training scripts and prepare the repository documentation and this write-up. I shared code and aggregate results with these assistants, not individual participants' results. I reviewed the outputs and am responsible for them. All training ran on my laptop. My Claude and OpenAI accounts are both set so that Anthropic/OpenAI do not use my conversations for model training.

10. How did you evaluate performance of the model other than the provided metric, if at all?

I tracked log loss and AUROC on the validation surfaces below and changed the pipeline only when a gain held on more than one of them.

- **Shared five-fold split.** Stratified five-fold cross-validation (seed 42) on 1,362 scans gave every member leak-free out-of-fold predictions for the stacker and calibration. The v17 ensemble, rebuilt from out-of-fold predictions, scores 0.2039 (AUROC 0.9751).
- **Gating rule.** After about 75 rounds of work on that split, I trusted no change below 0.005 of CV loss until it won on at least 5 of 6 random re-splits and 4 of 5 leave-group-out partitions. I judged new members by their effect on the full stack, since fold 0 alone would have rejected the 2D member.
- **Leave-site-out.** The data do not name acquisition centres. I inferred about 8 from 137 geometry signatures and raw intensity scale. Leaving one out moved `lin` from 0.2288 to 0.2400 and the v13c stack from 0.2046 to 0.2091. For the v17 tabular block I held out each of 6 acquisition families (0.2230) and ran a fresh split with seed 170919 (0.2163), against 0.2054 on the shared split.
- **Paired bootstrap.** Paired resampling of out-of-fold rows forecast -0.0037 (95% interval -0.0100 to +0.0018) for v13c against v11. The public board returned -0.0040.
- **Pre-registered forecasts.** For v10a (v7 at T = 1.2) I recorded 0.2575 ± 0.004 with AUROC unchanged at 0.9591. The board returned 0.2507 and 0.9591, so the loss beat the forecast by 0.0068. For v13c I registered 0.247-0.253 and 0.959-0.961, and it scored 0.2462 and 0.9606.

- **Drift laws.** Public AUROC equalled CV AUROC minus 0.008 to 0.014 on every scored version with a recorded public AUROC (v2 to v15a3). At T = 1 the CV-to-public loss drift followed 0.0312 + 1.34 × (0.2287 - CV loss). Public-to-private drift fell from +0.0377 (v7) to +0.0276 (v13c), then rose to +0.0306 (v15a3) and +0.0322 (v17), the two scored versions with the tabular block.
- **Prevalence tracking.** For gains that depended on acquisition, I compared the loss against an oracle per-stratum intercept. 69% of the head-volume feature's local gain was a per-stratum prevalence fit. I added this check after v9, which won on every local surface and lost on the public board.
- **Error analysis.** The two hardest inferred centres hold 36% of scans and 45% of CV loss. Four pipelines that share only the striatal localisation misclassify the same 25 scans (1.8%), which carry 19.5% of the loss.
- **Synthetic degradation.** I pushed blur, noise, gain changes, thick slices and field-of-view cuts through the whole pipeline (9,534 degraded scans, no hard failures). Under noise the stack held AUROC 0.947 on scans mislocalised by up to 2 mm and fell to 0.70 beyond 6 mm. The reject-to-prior governor cut the loss of the noise cell from 0.5648 to 0.4696.
- **Calibration and parity.** Observed and predicted rates matched in three probability bins (0.033/0.031, 0.453/0.474, 0.962/0.965 for v13c). Recomputing v17 features from native volumes changed held-out log loss by +1.6e-5.

11. What are some other things you tried that didn't necessarily make it into the final workflow (quick overview)?

- **Affine (7-DOF) registration:** better fit (NCC 0.834 against 0.788) and worse loss (0.2720 against 0.2430). The scale term registers the disease away.
- **Rejecting registrations rotated more than 30°:** cost 0.0066 of CV.
- **Registered-frame crops for the CNNs:** the 3D network lost 0.019 at fold 0 and the 2D member 0.010.
- **Head-volume and enclosed-void features (v8, v9):** 0.0041 better CV and 0.0155 better leave-site-out, then 0.2664 against 0.2611 on the public board.
- **Other crop geometries:** all equal or worse than 96 mm at 2 mm. 3 mm voxels cost 0.008.
- **More 3D seeds or epochs:** nine runs equalled six to four decimals. 60 epochs scored 0.045 worse than 20.
- **Pretrained 3D encoders:** masked-patch 0.2748, SimSiam 0.2714 and MedicalNet 0.3635-0.5518, against 0.2543 and 0.2592 for the matching runs from random initialisation.
- **Other 2D backbones:** ConvNeXt-nano 0.2965, ResNet-18 0.3010, 256 px input 0.2921, all with stacker weights of 0.000-0.026 next to B0. Frozen DINOv2 features scored 0.4058 and worsened the stack from 0.2090 to 0.2122.
- **LightGBM and MLP on the features:** 0.2716 and 0.2717 against 0.2509 for the L1 model on the same table.
- **Stacking variants:** tree and conditional stackers 0.2329-0.2427 against 0.2300, residual boosting 0.2139 to 0.2390, cross-fitted per-centre intercepts +0.00086.

- **Row trimming and hard-centre up-weighting:** trimming was worse in 22 of 22 tests, and up-weighting moved overall loss from 0.2322 to 0.2326-0.2354.
- **Other tabular learners:** TabPFN-2 0.2206, TabDPT 0.214, Mitra-2 0.2137, TabM 0.2362-0.2514, SVMs 0.226-0.269, CatBoost 0.292-0.309, TabICL fine-tuning 0.2186.
- **Other TabICL inputs:** image embeddings (0.2164-0.2314, no gain in the blend) and background-noise features (0.2526 with families held out).
- **Acquisition-randomised training rows:** mean leave-site-out 0.2742-0.2813 against 0.2696, with the worst site improving from 0.4435 to 0.3731.

12. If you were to continue working on this problem for the next year, what methods or techniques might you try in order to build on your work so far? Are there other fields or features you felt would have been very helpful to have?

Methods.

- **Calibration against drift.** v17 drifted +0.0322 from public to private against +0.0276 for v13c. I would test a higher tabular temperature.
- **Acquisition descriptors.** I would split their TabICL gain into discrimination and prevalence tracking with the oracle-intercept test.
- **Second-peak pairing.** 41 scans with a striatal separation above 50 mm have mean loss 0.342 against 0.212. I would pair the peaks, build the frame and re-pair inside it. At most 34 scans are at stake.
- **Localiser governor.** The distance between detections at 4 mm and 7 mm smoothing flags mislocalised scans under synthetic noise at AUC 0.71-0.75 with a 2% false-alarm rate on clean scans. I would use it to shrink flagged scans toward the prior.
- **CNN frame.** Registered crops gained the feature models 0.007 and cost the 3D CNNs 0.018-0.019 at fold 0 and the 2D reader 0.010. I would test whether the registration shift correlates with the label.
- **Registration.** Multi-start fitting cut CPU-to-GPU disagreements above 2° from 16 to 4 of 175 scans. I have not tried deformable registration, and the 7-DOF result warns that extra freedom can absorb the disease.
- **2D reader.** Two different backbones paid while extra seeds did not, so I would add a third architecture.

Data and fields.

- **More scans.** The learning curve still descends. Doubling the cohort should gain about 0.013-0.025 on the L1 model and 0.008-0.015 on the stack. Monte Carlo simulation (SIMIND or GATE) is the only licence-clean source I found, and I estimated it at weeks of work.
- **Acquisition metadata.** The NIfTI headers carry no acquisition fields. Centre, camera, collimator, reconstruction settings, and attenuation and scatter

correction would let the models separate acquisition from disease in the two hardest inferred centres, which carry 45% of CV loss.

- **Age and sex.** The data contain neither. Head-mask volume partly stands in for sex.
- **Follow-up diagnosis.** A later clinical diagnosis would add information that a single scan cannot give. It would help most with the 25 scans that all four pipelines misclassify.
- **Label protocol.** I would like to know how the organisers assigned labels: visual read or semi-quantitative software, how many readers, and their agreement.

13. What simplifications could be made to run your solution faster without sacrificing significant accuracy?

- **Fewer 3D checkpoints (34 to about 10).** Seed averaging saturates: a two-run member scores 0.2617 against about 0.2600 for six runs, and nine runs equal six to four decimals. The full-data checkpoints made no visible difference on the board (AUROC 0.9591, 0.9591 and 0.9589 for v7, v10a and v11).
- **Without the EfficientNet-B3 member.** The 150-scan local run fell from 9 min 14 s to 7 min 3 s (24%) at a local loss cost of about 0.0008. I never scored this variant.
- **No 2D flip test-time augmentation.** Removing the flip halves 2D inference compute. Flip and scale test-time augmentation together changed the stack by 0.000. I did not measure flip alone.
- **Without the TabICL block (v13c).** Private log loss 0.2738 against 0.2731, and public 0.2462 against 0.2409. The archive shrinks from 1,090 MB to 292 MB, and the 150-scan local run took 6 min 24 s against 530 s on a different device setup.
- **Smaller TabICL.** Four estimators scored 0.0004 worse than sixteen in a v15-era blend, so four in place of eight should cost less. 48 selected features in place of 64 improved the blend by 0.0005 (0.2018 against 0.2023).
- **Shorter registration.** 60 and 40 optimisation steps in place of 120 and 80 gave equal CV in about half the time.
- **Parallel preprocessing.** Preprocessing takes about 0.63 s per scan on one core, and the loop runs serially. Spreading scans over the container's 24 vCPUs costs no accuracy.
- **CPU only.** At 3.3-6.8 s per scan on the M3 Pro CPU, a 1,000-scan test set would take about 1-1.9 h on a similar CPU, within the 3 h limit. This is an estimate from the 150-scan runs.